

I145 TALLER DE LENGUAJE .NET

Fase 1: Dominio Rico y Arquitectura Limpia

Sistema para la Gestión de ExpedientesAvila Montoya, Eygleen Fernanda¹ (02931/2)Bejarano, Abril¹ (03339/5)Tarifa, Armando Ezequiel² (02893/4)

19/05/2026

¹{eygleen.avila, abril.bejarano}@alu.ing.unlp.edu.ar²eze.tarifa.2002@gmail.com**1. Prueba de funcionalidad desde Program.cs**

El archivo `Program.cs` contiene las instancias de los repositorios, servicios y casos de uso necesarios, así como también el código de ejemplo para ambos caminos del programa: el caso en el que todo funciona correctamente y el caso donde se provocan los errores esperados para verificar el manejo de excepciones.

1.1. Configuración inicial

Al inicio del programa se crean las instancias de los repositorios y servicios que utilizarán todos los casos de uso:

```
IExpedienteRepository expedienteRepo = new ExpedienteTxtRepository();
ITramiteRepository tramiteRepo = new TramiteTxtRepository();
IAutorizacionService authService = new AutorizacionProvisionalService();

var actualizacionService = new ActualizacionEstadoExpedienteService(expedienteRepo,
    tramiteRepo);

var agregarExpUseCase = new AgregarExpedienteUseCase(expedienteRepo, authService);
var listarExpUseCase = new ListarTodosLosExpedientesUseCase(expedienteRepo);
var modificarCaratulaUseCase = new ModificarCaratulaExpedienteUseCase(expedienteRepo,
    authService);
var cambiarEstadoUseCase = new CambiarEstadoExpedienteUseCase(expedienteRepo, authService);
var eliminarExpUseCase = new EliminarExpedienteUseCase(expedienteRepo, tramiteRepo,
    authService);
var agregarTramiteUseCase = new AgregarTramiteUseCase(tramiteRepo, authService,
    actualizacionService);
var listarTramitesUseCase = new ListarTramitesPorExpedienteUseCase(tramiteRepo);
var modificarTramiteUseCase = new ModificarTramiteUseCase(tramiteRepo, authService,
    actualizacionService);
var eliminarTramiteUseCase = new EliminarTramiteUseCase(tramiteRepo, authService,
    actualizacionService);

Guid miUsuario = Guid.NewGuid();
```

1.2. Camino feliz**1.2.1. Agregar un expediente**

Se crea un expediente con carátula "Solicitud de Hardware". El caso de uso devuelve el expediente creado con su ID asignado.

```
var res = agregarExpUseCase.Ejecutar(new AgregarExpedienteRequest("Solicitud de Hardware",  
miUsuario));  
idExpediente = res.Id;
```

Salida esperada:

```
-> Agregando expediente 'Solicitud de Hardware'...  
[OK] Expediente creado. ID: aa6ca6dc-4067-4908-8ed5-0c2ceca4b623
```

1.2.2. Agregar trámite PaseAEstudio

Se agrega un trámite con etiqueta PaseAEstudio. Esto dispara el servicio de actualización de estado, que cambia el estado del expediente a ParaResolver.

```
agregarTramiteUseCase.Ejecutar(new AgregarTramiteRequest(  
    idExpediente,  
    EtiquetaTramite.PaseAEstudio,  
    "Se evalúa el presupuesto disponible.",  
    miUsuario));
```

Salida esperada:

```
-> Agregando trámite 'PaseAEstudio' (estado debería pasar a ParaResolver)  
[OK] Trámite agregado.
```

1.2.3. Agregar trámite Resolución

Se agrega un trámite con etiqueta Resolucion. El servicio de actualización vuelve a ejecutarse y cambia el estado del expediente a ConResolucion.

```
agregarTramiteUseCase.Ejecutar(new AgregarTramiteRequest(  
    idExpediente,  
    EtiquetaTramite.Resolucion,  
    "Se aprueba la solicitud.",  
    miUsuario));
```

Salida esperada:

```
-> Agregando trámite 'Resolucion' (estado debería pasar a ConResolucion)  
[OK] Trámite agregado.
```

1.2.4. Listar expedientes

Se listan todos los expedientes registrados para verificar que el estado se haya actualizado correctamente.

```
var lista = listarExpUseCase.Ejecutar(new ListarTodosLosExpedientesRequest());  
foreach (var dto in lista.Expedientes)  
{  
    Console.WriteLine($"    Carátula: {dto.Caratula} | Estado: {dto.Estado} | ID: {dto.Id}");  
}
```

Salida esperada:

```
-> Listando todos los expedientes  
    Carátula: Solicitud de Hardware | Estado: ConResolucion | ID:  
aa6ca6dc-4067-4908-8ed5-0c2ceca4b623
```

1.2.5. Listar trámites del expediente

Se listan los trámites asociados al expediente creado para confirmar que ambos fueron persistidos correctamente.

```
var tramites =  
listarTramitesUseCase.Ejecutar(new ListarTramitesPorExpedienteRequest(idExpediente));  
foreach (var t in tramites.Tramites)  
{
```

```
Console.WriteLine($" Trámite: {t.Etiqueta} | Contenido: {t.Contenido} | ID: {t.Id}");  
}
```

Salida esperada:

-> Listando trámites del expediente
Trámite: PaseAEstudio | Contenido: Se evalúa el presupuesto disponible. | ID: 6f2e6b5c-c13b-4fbf-86ce-5927e064d600
Trámite: Resolucion | Contenido: Se aprueba la solicitud. | ID: 5f61bc30-8b0b-48fb-b129-240e32129332

1.2.6. Cambio de estado manual

Se prueba el cambio de estado manual del expediente, pasándolo a `EnNotificacion`, independientemente de los trámites.

```
cambiarEstadoUseCase.Ejecutar(new CambiarEstadoExpedienteRequest(  
    idExpediente,  
    EstadoExpediente.EnNotificacion,  
    miUsuario));
```

Salida esperada:

-> Cambiando estado manualmente a 'EnNotificacion'
[OK] Estado cambiado.

1.3. Camino triste

En este caso, se prueban situaciones de error para verificar que las excepciones del dominio y de autorización se lanzan y capturan correctamente.

1.3.1. Error 1: Carátula vacía

Si se intenta crear un expediente con carátula vacía, el dominio lanza una `DominioException`.

```
agregarExpUseCase.Ejecutar(new AgregarExpedienteRequest("", miUsuario));
```

Salida esperada:

-> Intentando crear expediente con carátula vacía
[CAPTURADO - DOMINIO]: La carátula no puede estar vacía ni ser nula.

1.3.2. Error 2: Contenido de trámite vacío

Si se intenta agregar un trámite sin contenido, también se lanza una `DominioException`.

```
agregarTramiteUseCase.Ejecutar(new AgregarTramiteRequest(  
    idExpediente,  
    EtiquetaTramite.Despacho,  
    "",  
    miUsuario));
```

Salida esperada:

-> Intentando agregar trámite con contenido vacío
[CAPTURADO - DOMINIO]: El contenido del trámite es obligatorio y no puede estar vacío.

1.3.3. Error 3: Expediente inexistente

Si se intenta modificar la carátula de un expediente que no existe (usando un `Guid` aleatorio), se lanza una `DominioException`.

```
modificarCaratulaUseCase.Ejecutar(new ModificarCaratulaExpedienteRequest(  
    Guid.NewGuid(),  
    "Nueva carátula",  
    miUsuario));
```

Salida esperada:

-> Intentando modificar carátula de un expediente inexistente
[CAPTURADO - DOMINIO]: No se encontró el expediente solicitado.

1.3.4. Error 4: Autorización denegada

Para probar el caso de `AutorizacionException`, se debe ir a la clase `AutorizacionProvisionalService` y cambiar el `return true` por `return false`. Al hacerlo, cualquier operación que requiera autorización lanzará esta excepción.

```
// En AutorizacionProvisionalService:  
public bool EstaAutorizado(Guid usuario, Permiso permiso) => false;
```

Salida esperada (al intentar agregar un expediente, por ejemplo):
[CAPTURADO - AUTORIZACION]: Usuario no autorizado para agregar expedientes.

1.4. Salida completa de la consola

Esta es la salida completa que se espera después de ejecutar `Program.cs` con `AutorizacionProvisionalService` en `return true`:

CAMINO FELIZ :) :

SGE - SISTEMA DE GESTION DE EXPEDIENTES

-> Agregando expediente 'Solicitud de Hardware'...

[OK] Expediente creado. ID: aa6ca6dc-4067-4908-8ed5-0c2ceca4b623

-> Agregando trámite 'PaseAEstudio' (estado debería pasar a ParaResolver)

[OK] Trámite agregado.

-> Agregando trámite 'Resolucion' (estado debería pasar a ConResolucion)

[OK] Trámite agregado.

-> Listando todos los expedientes

Carátula: Solicitud de Hardware | Estado: ConResolucion | ID:
aa6ca6dc-4067-4908-8ed5-0c2ceca4b623

-> Listando trámites del expediente

Trámite: PaseAEstudio | Contenido: Se evalúa el presupuesto disponible. | ID: 6f2e6b5c-c13b-4fbf-86ce-5927e064d600

Trámite: Resolucion | Contenido: Se aprueba la solicitud. | ID: 5f61bc30-8b0b-48fb-b129-240e32129332

-> Cambiando estado manualmente a 'EnNotificacion'

[OK] Estado cambiado.

CAMINO TRISTE :(:

-> Intentando crear expediente con carátula vacía

[CAPTURADO - DOMINIO]: La carátula no puede estar vacía ni ser nula.

-> Intentando agregar trámite con contenido vacío

[CAPTURADO - DOMINIO]: El contenido del trámite es obligatorio y no puede estar vacío.

-> Intentando modificar carátula de un expediente inexistente

[CAPTURADO - DOMINIO]: No se encontró el expediente solicitado.

-> (Para probar `AutorizacionException`: cambiar `AutorizacionProvisionalService` a `return false`)
Al hacerlo, cualquier operación lanzará `AutorizacionException`.

FIN DE LA DEMO